

Ime i prezime: _____

Bodovi

Question:	1	2	3	4	5	6	7	Total
Points:	17	2	6	6	9	6	4	50
Score:								

1. U ovom pitanju, radit ćete se sa OCaml programskim jezikom kako biste implementirali funkcije koje rade sa fajl sistemom računara (folderi i fajlovi). Tip koji predstavlja fajl sistem dat je ispod. Tip **Fajl** sačinjavaju naziv fajla i njegova veličinu u bajtima, a tip **Direktorij** sačinjavaju naziv direktorija i sadržaj tog direktorija.

```
type fajl_sistem =  
  | Fajl of string * int  
  | Direktorij of string * fajl_sistem list
```

- (a) (/2 points) Definišite instancu tipa `fajl_sistem` koja predstavlja sljedeću strukturu:

- Direktorij nazvan `root` koji sadrži:
 - Fajl nazvan `file1.txt` veličine 100 bajtova.
 - Direktorij nazvan `subdir` koji sadrži:
 - * Fajl nazvan `file2.txt` veličine 200 bajtova.
 - * Fajl nazvan `file3.txt` veličine 300 bajtova.
 - * Direktorij nazvan `prazan`.

- (b) (/2 points) Napišite rekurzivnu funkciju `ukupna_velicina` koja računa ukupnu veličinu svih fajlova u datom fajl sistemu.

- (c) (/2 points) Napišite rekurzivnu funkciju `pronadji_fajl` koja provjerava da li fajl sa datim imenom postoji u fajl sistemu.

- (d) (/1 point) Proširite tip `fajl_sistem` da uključuje simboličke linkove, gdje simbolički link ima ime i pokazuje na drugi fajl ili direktorij.

- (e) (/1 point) Definišite instancu proširenog tipa `fajl_sistem` koja predstavlja sljedeću strukturu:

- Direktorij nazvan `novi_root` koji sadrži:
 - Fajl nazvan `file0.txt` veličine 50 bajtova.
 - Symlink nazvan `sub_link` koji sadrži pokazuje na fajl sistem definisan u dijelu (a) zadatka.

- (f) (/3 points) Napišite rekurzivnu funkciju `prati_symlinkove` koja za date `fajl_sistem` i `ime`, vraća element tipa `Some fajl_sistem` na koji simbolički link pokazuje. Ako `ime` nije simbolički link, funkcija vraća `None`.

3. Ovo pitanje razmatra razmijevnaje zatvaranja funkcija (eng. function closure).

- (a) (/2 points) Implementirajte funkciju `napravi_sabirac` koja prima jedan argument `n` i vraća novu funkciju. Nova funkcija treba primiti jedan argument `x` i vratiti rezultat zbrajanja `n` i `x`.

- (b) (/1 point) Navedite primjer pozivanja funkcije `napravi_sabirac` i pozivanja njenog rezultat, kao i očekivani rezultat tog poziva.

- (c) (/1 point) Objasnite gdje je u prethodnim primjerima demonstrirana upotreba closure-a.

- (d) (/2 points) Koja je vrijednost varijable `r1` u kodu ispod. Opišite kako ste došli do tog rezultata i gdje se izračun oslanja na closure.

```
let m = 2
let n = 3
let p = 4
let q = 5
let combine (f, g, h) =
  let m = 10 in
  f m + g n + h p

let m = 7
let n = 8
let r1 = combine ((fun x -> m * x), (fun y -> y + n), (fun z -> z * q))
```

4. Streams. Kao što smo definisali na času, tok (eng. stream) je odgođeni izračun (eng. thunk) koji, kada se pozove, vraća par čiji je cdr (rep) također tok.

- (a) (/1 point) Napisati funkciju `dole-od-n` koja uzima cjelobrojni broj `n` i vraća tok (eng. stream) gdje je prvi element `n`, sljedeći element je `n-1` i tako dalje beskonačno.

- (b) (/1 point) Napisati funkciju `gore-od-n` koja uzima cjelobrojni broj `n` i vraća tok (eng. stream) gdje je prvi element `n`, sljedeći element je `n+1` i tako dalje beskonačno.

- (c) (/2 points) Napisati funkciju `nekud-od-n` koja uzima dva parametra: operator i *broj*. Funkcija vraća stream gdje se krenuvši od broja nove vrijednosti generišu primjenom operatora nad brojem (i svakim narednim brojem) i vrijednosti 1.

- (d) (/2 points) Upotrebom parcijalne primjene funkcije `nekud-od-n` ponovo kreirajte `dole-od-n` i `gore-od-n`.

5. U nastavku je data klasa `c%` koju je potrebno razmotriti.

```
(define c%  
  (class object%  
    (super-new)  
    (define a 5)  
    (define/public (foo x y) (+ x y 5))  
    (define/public (bar z) (send this foo z z))))
```

Definisane su 3 izvedene klase za koje je potrebno napisati rezultat kada se njihovoj metodi proslijedi vrijednost 10. U slučaju da dolazi do greške opisati njen uzrok.

- (a) (/1 point) Klasa `d%`

```
(define d%  
  (class c%  
    (super-new)  
    (define/public (baz z) (+ a z))))
```

- (b) (/1 point) Klasa e%

```
(define e%  
  (class c%  
    (super-new)  
    (define/override (foo z) (+ z 5))))
```

- (c) (/1 point) Klasa f%

```
(define f%  
  (class c%  
    (super-new)  
    (define/override (baz z) (send this foo (+ z z) z))))
```

- (d) (/2 points) Objasnite šta je mixin, kada se koristi i zašto je Racket programski jezik pogodan za njegovo kreiranje.

- (e) (/2 points) Napisati mixin **dodavac-baterije** koja u proslijeđenu klasu dodaje novo polje naziva **status-baterije** kao i getter i setter za ovo polje.

- (f) (/2 points) Napisati kod koji demonstrira upotrebu mixina iznad nad klasom **c%** sa početka ovog zadatka. Demonstrirajte primjenu mixin-a, kreiranje objekta, kao i upotrebu gettera i settera nad tim objektom.

6. Dinamičko otpremanje.

(a) (/2 points) Objasnite koncept dinamičkog otpremanja (eng. dynamic dispatch).

(b) (/4 points) Kreiranjem klasa i objekata u programskom jeziku Racket, te pozivima njihovih metoda demonstrirajte funkcionisanje dinamičkog otpremanja. U polju ispod naznačite upotrebu mehanizma. Nije neophodno da kreirate smislene klase (npr. osoba, računar) već tehnički ispravne.

7. Sistem tipova.

- (a) (/2 points) Da li je su programski jezici OCaml i Racket pravilni (eng. sound) i potpuni (eng. complete)? Pojasnite.

- (b) (/2 points) Objasnite pojam slabog tipiziranja (eng. weak typing).
